

Cleaner: To-Do with AI 项目报告

组员:

- 袁野 10235101541
- 陈磊 10235101567
- 龙胜同 10235101479

分工:

成员	职责
袁野	基础功能、系统优化
陈磊	系统优化、演示 PPT 制作
龙胜同	系统优化、实验报告

一、项目内容概述

1.1 简介

Cleaner — AI 待办管理是一款基于 Flet 框架开发的跨平台桌面待办事项管理应用。项目核心目标是打造一个智能任务管理系统，融合 AI 语义理解、本地数据持久化、丰富的可视化统计和完善的人机交互体验。

1.2 需求分析

目标用户画像

- 脑力工作者**: 追求极致效率与严格规划，需要同时记录管理多时间段的事务。如学生记录管理课业与考试、职场办公人员记录管理会议等。
- 长期主义生活/打卡**: 面临定时定点周期性重复任务，记录长期积累成果。如进行业余爱好自学规划，生活场景定期规划。

传统待办软件痛点

1. 用户需要手动填写软件中的各种栏目，需要拆分思考任务结构再录入任务，存在额外的思考成本与录入成本，甚至超过了记录想法本身的成本。
2. 自然语言无法直接转化为任务，用户会倾向于只记录自然语言，而直接忽略软件中提供的其他功能（如日期、时间、分级等）。
3. 市面上目前待办工具的 AI 功能强依赖网络，离线环境则 AI 功能不可用。

功能性需求

- **任务创建**：用户能够创建待办事项，包括传统输入和自然语言输入。
- **任务列表管理**：查看任务、编辑任务、删除任务、完成任务。
- **任务状态**：在任务卡片上显示基于当前时间的变色进度条。
- **AI 任务解析与管理**：利用 LLM 理解自然语言，转化为待办任务相关操作。
- **数据持久化**：采用 SQLite，保存任务信息、创建时间、完成状态。
- **搜索与筛选**：按多种条件筛选。
- **统计分析**：总任务数、完成率、时间段完成数等统计数据。
- **数据可视化**：提供任务视图和日历视图，对统计数据以多种图表形式呈现。

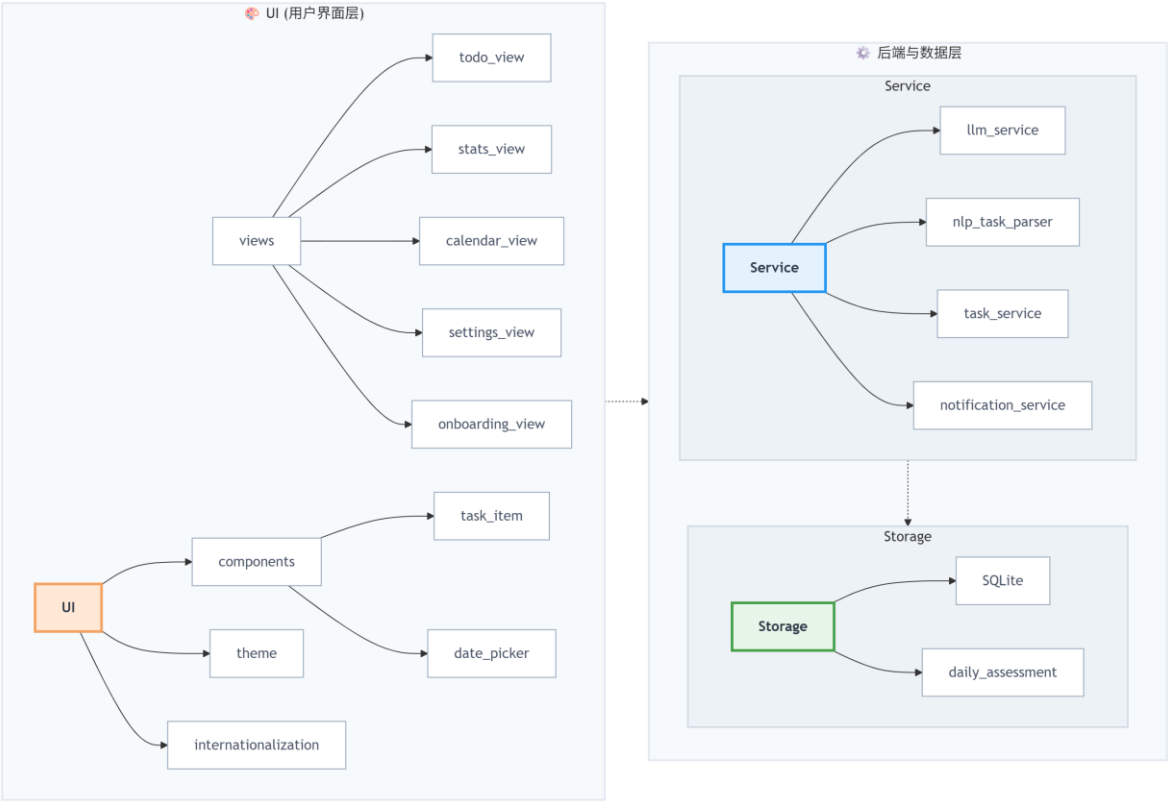
非功能性需求

- **易用性**：成年用户无需培训，可自主掌握大部分功能，可完成待办管理基本流程。
- **响应性能**：普通操作反馈 <200ms。
- **可用性**：AI 服务网络异常时会自动降级并完成服务。
- **数据可靠性**：异常退出后数据不丢失。
- **安全性**：用户数据不上传外部服务器，LLM 服务部分仅上传任务描述。
- **可维护性**：UI 层、Service 层、Storage 层分层结构，便于扩展和维护。

使用场景

- **日常规划**：用户打开电脑，双击打开 Cleaner，Cleaner 软件自动出现在上次用户所拖动到达的位置，用户可以自行创建任务进行管理。
- **自然语言输入**：用户在工作中临时收到工作任务，将工作任务文本直接复制粘贴到软件 AI 对话区域，自动拆分任务生成待办事项。
- **复盘整理**：用户可以通过 AI 对话或者点击 AI 快捷气泡，进行待办任务相关操作。切换到统计页面可查看任务完成情况，获得心理成就感。

1.3 系统架构



二、实现功能

2.1 AI 智能对话系统

功能	描述
自然语言操作	用中文对话新增、修改、删除、完成、查询、规划任务
意图识别	调用 LangChain + DeepSeek 进行工具选择和意图规划，输出结构化数据
Markdown 渲染	AI 回复使用 ft.Markdown 渲染，文字可选中
AI 记忆系统	持久化最近 10 轮对话，重启不丢失
思考动画	AI 处理期间显示 ProgressRing + "思考中"提示，通过 <code>asyncio.to_thread</code> 非阻塞执行
快捷气泡	AI 对话输入框上方提供快捷操作气泡

预设性格	5 种 AI 人设（阿喵/阿汪/砖家/小冰/默认），设置页一键切换
NLP 离线回退	无 API Key 或 LLM 服务失败时，自动回退到正则解析器

基本界面、快捷气泡



思考动画



意图识别、AI 回复



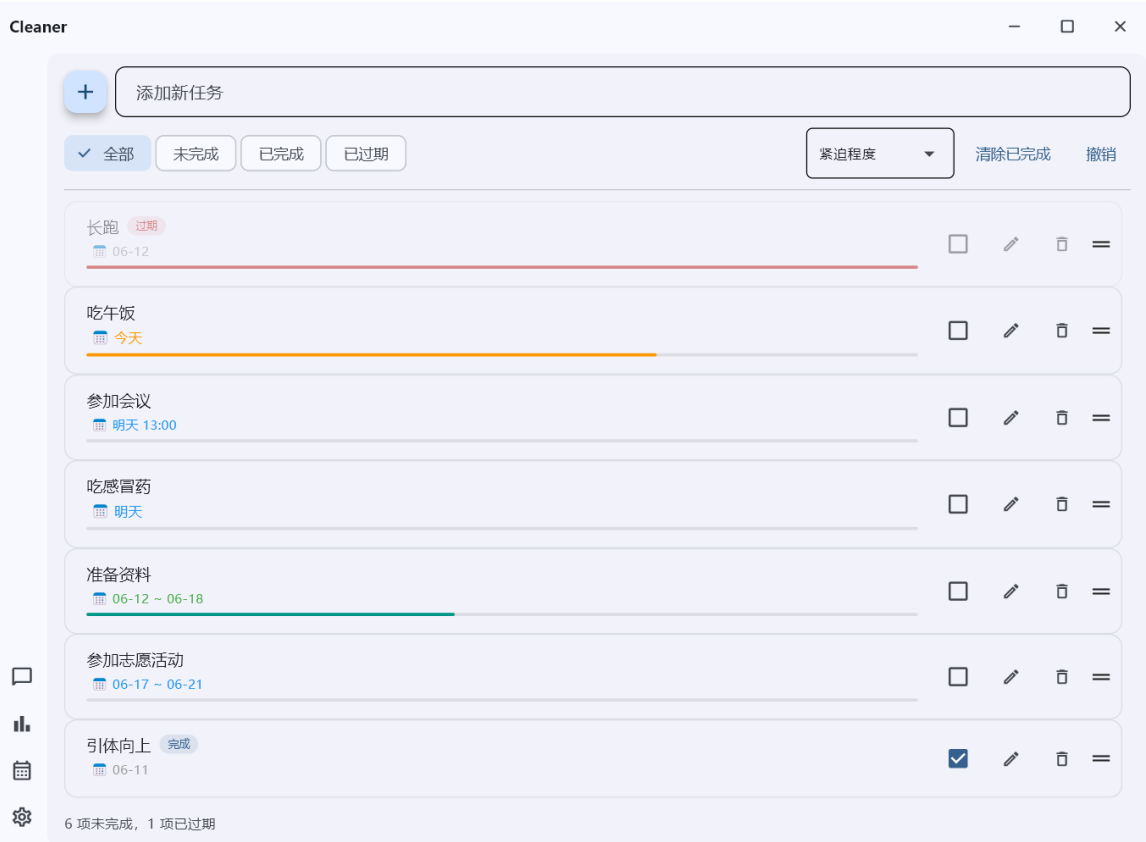
AI 配置、预设性格



2.2 任务管理

功能	描述
创建任务	支持输入名称、描述、开始日期/时间、结束日期/时间、重复模式
编辑任务	点击日期内联编辑，支持修改名称、描述、日期、时间、重复设置
完成/取消	Checkbox 勾选切换完成状态
拖拽排序	支持拖动卡片调整顺序
删除确认	删除需二次确认
撤销操作	最多保留 20 步操作快照
筛选	全部 / 未完成 / 已完成 / 已过期 四档筛选
排序	7 种排序：紧迫程度 / 日期↑ / 名称 A-Z Z-A / 持续时间↑
重复任务	支持"每天""隔天""每 N 天"重复，每期独立打卡，显示进度

基本界面、任务状态



创建待办任务

Cleaner

+ 添加新任务

< 2026年 6月 >

2026-06-14

点击选择日期，再点另一个变为持续

时间00:00 或 HH:MM

添加描述 (可选)

全部 未完成 已完成 已过期

紧迫程度

清除已完成 撤销

长跑 过期 06-12

吃午饭 今天

参加会议 明天 13:00

咕威咕咕

6 项未完成, 1 项已过期

指定任务重复模式

Cleaner

+ 添加新任务

今天 ~ 06-18

< 2026年 6月 >

2026-06-14 ~ 2026-06-18

点击选择日期，再点另一个变为持续

06-14 00:00

06-18 00:00

重复

不重复

每天

隔天

每3天

每7天

自定义

添加描述 (可选)

全部 未完成 已完成 已过期

紧迫程度

清除已完成 撤销

长跑 过期 06-12

吃午饭 今天

参加会议 明天 13:00

咕威咕咕

6 项未完成, 1 项已过期

编辑任务

Cleaner

+ 添加新任务

✓ 全部

未完成

已完成

已过期

紧迫程度

清除已完成

撤销

长跑

添加描述 (可选)

到期时间

23

:

00

=

重复

✓ 不重复

每天

隔天

每3天

每7天

自定义

吃午饭

今天

参加会议

明天 13:00

吃感冒药

明天

准备资料

06-12 ~ 06-18

6 项未完成, 1 项已过期

筛选、排序

Cleaner

+ 添加新任务

全部

✓ 未完成

已完成

已过期

名称 A-Z

清除已完成

撤销

准备资料

06-12 ~ 06-18

参加会议

明天 13:00

参加志愿活动

06-17 ~ 06-21

吃午饭

今天

吃感冒药

明天

长跑

06-12

名称 A-Z

紧迫程度

日期 ↓

日期 ↑

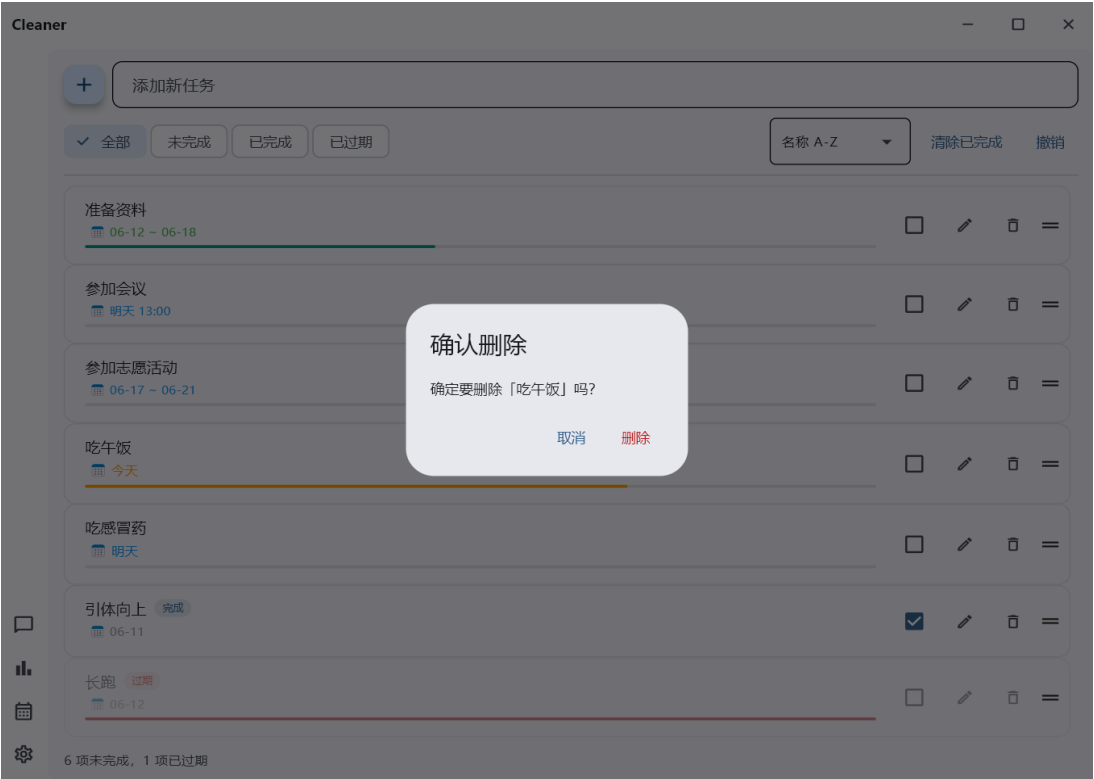
名称 A-Z

名称 Z-A

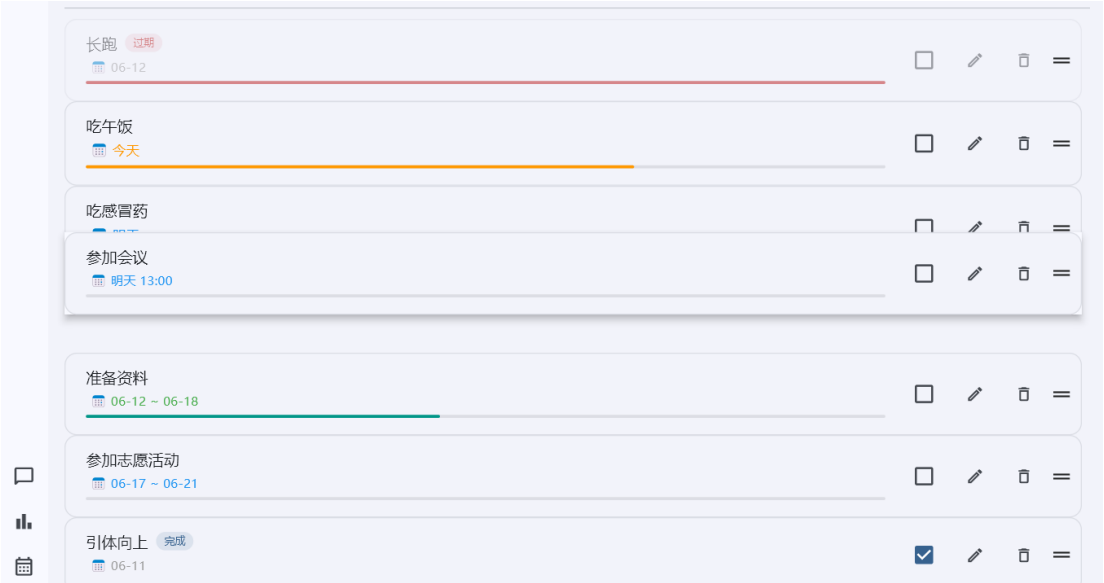
持续时间 ↓

持续时间 ↑

二次确认



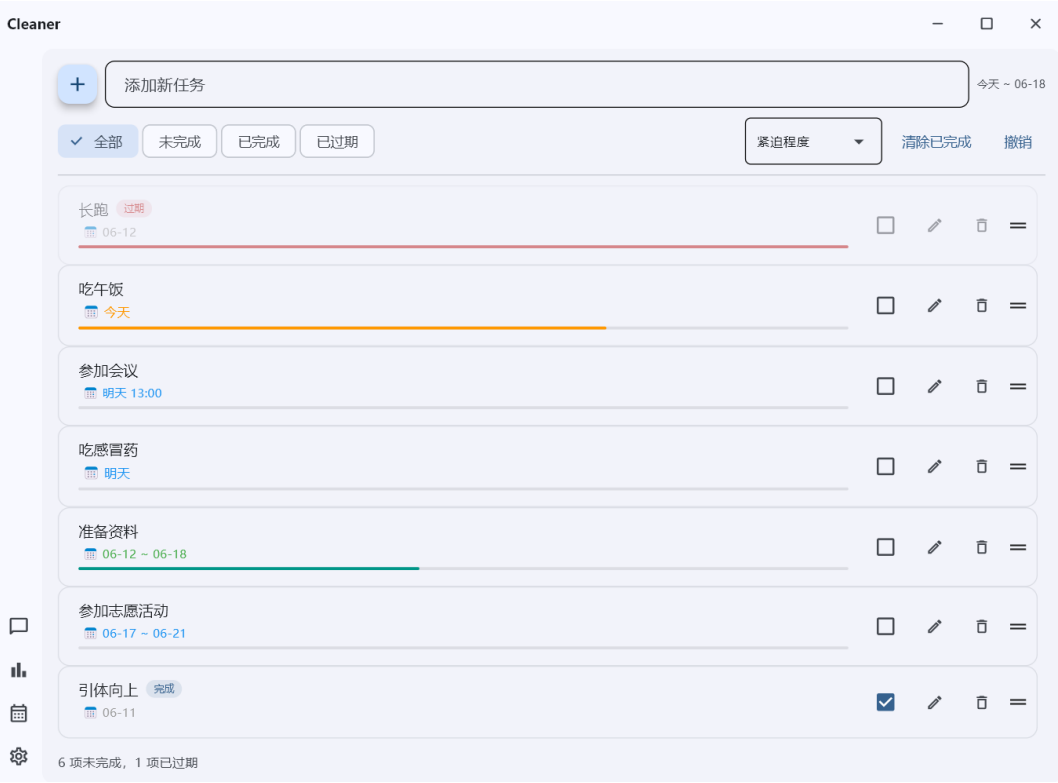
拖拽卡片



2.3 任务卡片

功能	描述
持续时间	支持跨天持续任务
精确时间	精确到小时/分钟
日期颜色编码	今天（橙）、未来（蓝）、过期（灰）、正在持续（绿）
卡片状态标记	已完成（加深背景+"完成"标签）、已过期（透明+"过期"标签）、进行中（绿色时间戳）
时间进度条	持续任务显示进度条
默认到期时间	可配置默认到期时间

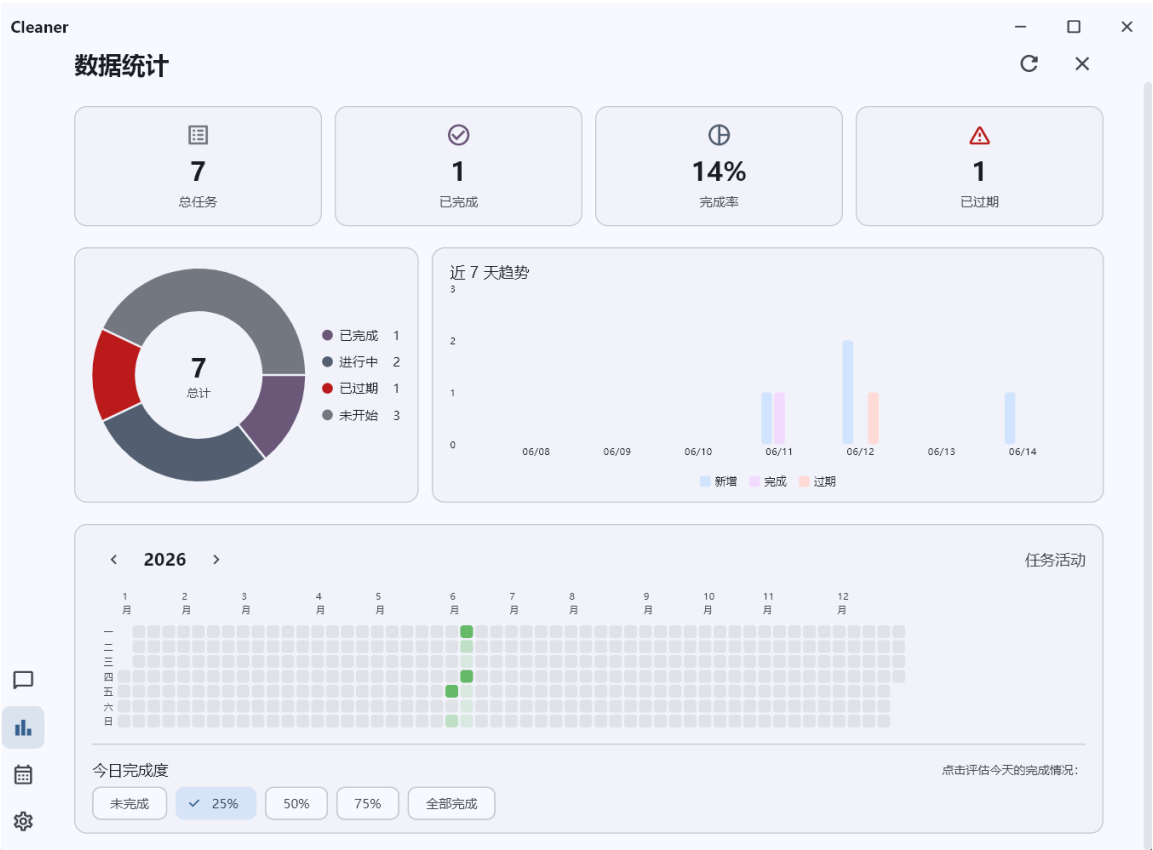
任务卡片展示



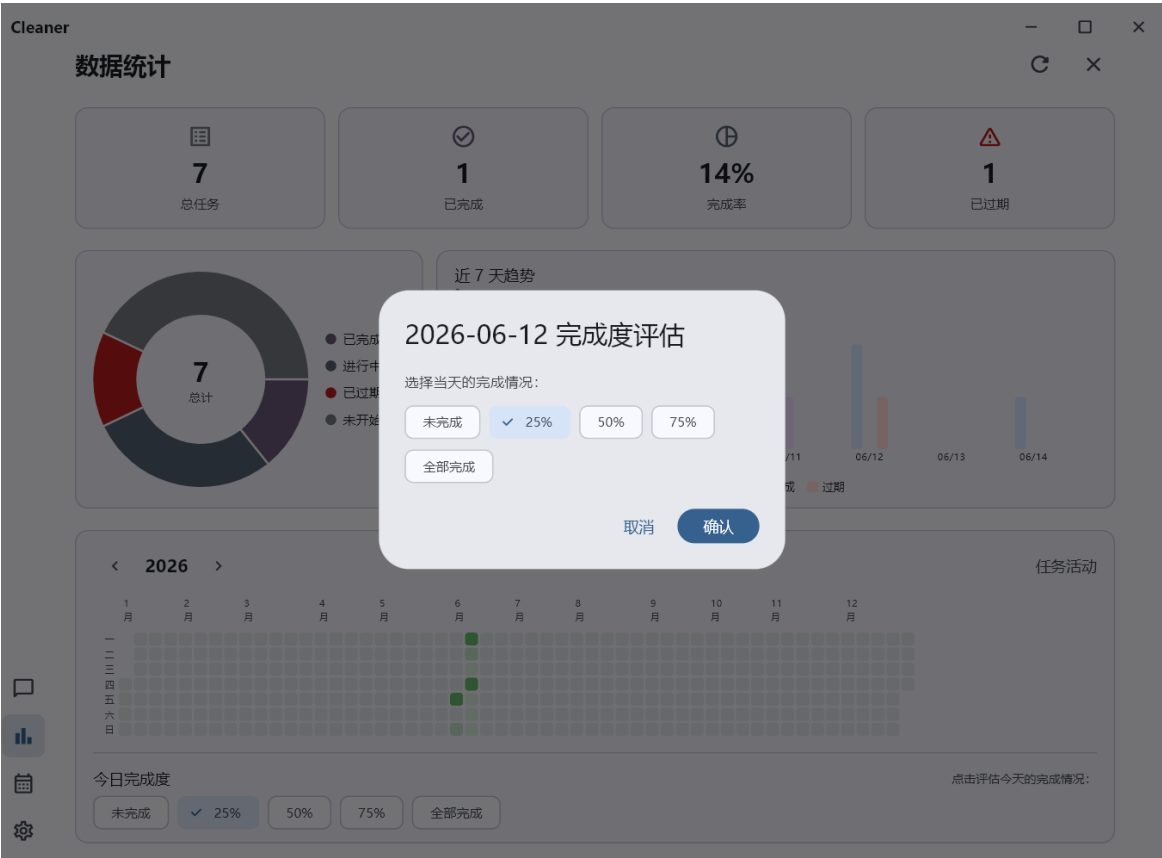
2.4 数据统计

功能	描述
概览卡片	4 张卡片（总任务 / 已完成 / 完成率 / 已过期）
环状图	4 段（已完成/进行中/已过期/未开始），中心叠加总数
7 天趋势	每日 2 柱（新增/完成），含日期轴和数值轴
热力图	全年 365 天完成度方块，5 级透明度阶梯，未来日期浅灰底色
每日评估	用户可手动评估每日完成度，自动回填历史日期
年份切换	热力图支持年/月导航，历史日期点击弹出判定对话框

统计图例



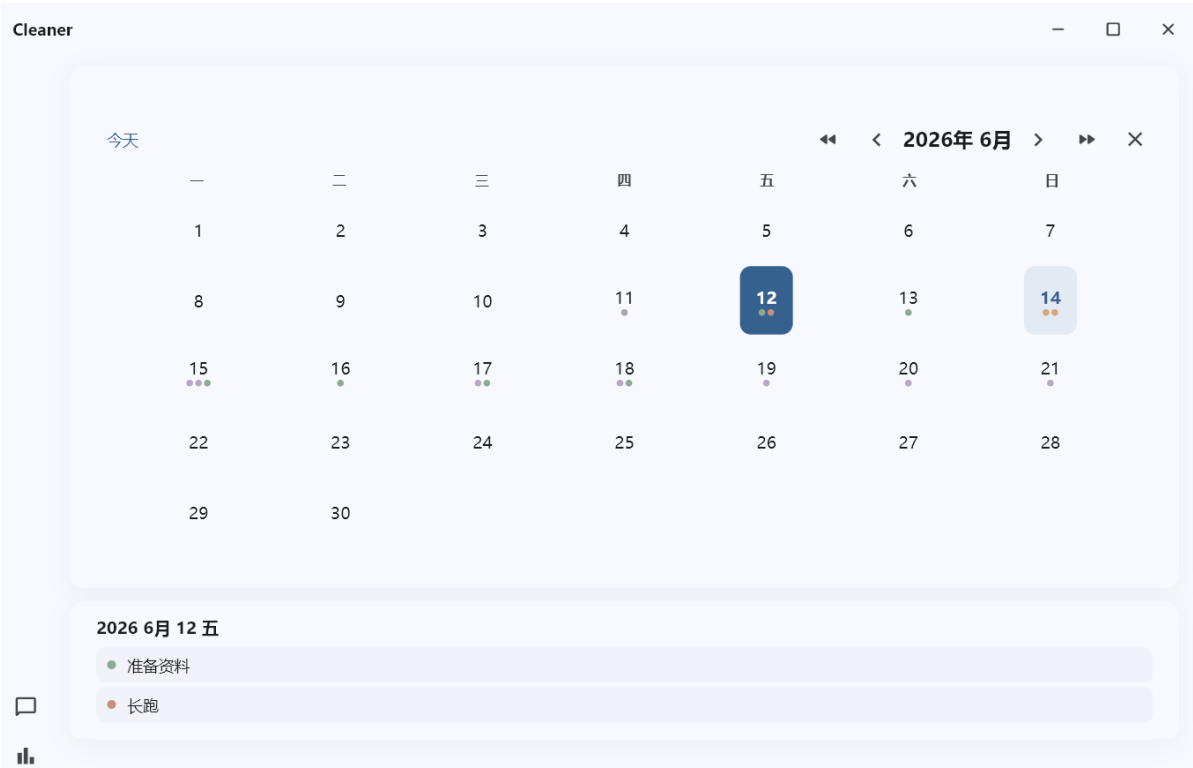
手动评估



2.5 日历视图

功能	描述
月网格	7 列×5-6 行表格，日期数字+任务圆点
状态区分	6 色圆点区分状态
年/月导航	<< >> 切换年份， < > 切换月份， "今天"快速导航按钮
详情面板	点击日期显示当天任务详细信息列表
重复任务适配	每日模式在每个应完成日期显示独立圆点

日历视图展示



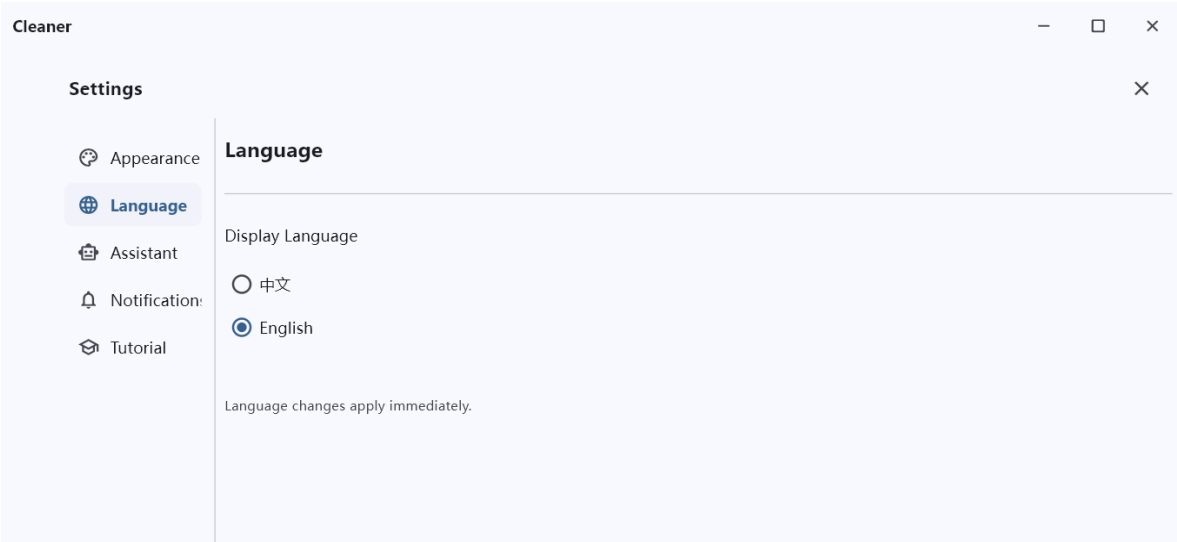
2.6 外观与个性化

功能	描述
主题模式	浅色 / 深色 / 跟随系统 三选一
主题色	6 种预设（蓝/靛/紫/青/橙/粉）
多语言	中文/英文界面切换，190+ 翻译键，实时刷新无需重启
窗口记忆	位置、大小、最大化/全屏状态持久化，多显示器智能检测
侧边栏	智能助手 / 统计 / 日历 / 设置 图标按钮，选中态背景高亮

外观设置



语言设置



2.7 系统通知

功能	描述
Windows 通知	基于 winotify 发送原生通知（重复打卡提醒、到期提醒、过期提醒）
免打扰	可配置免打扰时段（如 23:00–08:00），支持跨午夜
去重机制	每任务每天每种类型只通知一次
提前提醒	可配置 5/15/30/60 分钟提前量

启动时通知



通知设置



2.8 引导教程

功能	描述
7 步教程	欢迎 → 功能概览 → AI 服务 → 任务管理 → 视图通知 → API 配置 → 完成
重新查看	设置页"教程"入口随时重新打开

入门教程



重新查看



三、人机交互设计

3.1 可用性目标

效率

输入效率：通过 AI 聊天侧边栏，将传统的"标题+日期+时间+循环规则"多步骤表单输入，降维为一句话自然语言输入。配合快捷指令气泡（如"接下来做什么"），使用户的操作耗时降至最低。

安全性

- **防止误操作**：删除任务时要求二次确认。提供撤销功能，给予用户操作容错空间。
- **操作降级**：当断网或 LLM API 失效时，系统通过本地 NLP 正则解析器无缝接管，确保应用不会停止服务或产生空白响应。
- **多显示器协同**：当副屏断开时，系统自动将窗口拉回主屏，避免软件不可用。

效用性

- **任务类型细分**：精准区分不同类型、不同标签的待办任务。如对于长期任务，分别实现"一次即完成"和"多次独立完成"的不同需求。
- **任务全面操作**：支持任务的创建、编辑、删除、完成、统计的全面操作，覆盖任务全生命周期。

易学性

传统软件需要用户完全理解新建-设置-分类管理的全流程。在本软件中用户在上手阶段只需要输入自然语言文本，即可自动完成任务操作。

易记性

- **状态持久化**：用户离开后再次打开，窗口保持在原有位置和大小，修改过的设置保持不变。
- **上下文持久化**：AI 拥有多轮持久化记忆，减少了用户重构情境的成本。
- **AI 对话**：对话聊天的方式与日常语言或社交媒体使用方法一致。

3.2 交互类型

类型	说明
指示	包括创建、修改、删除、完成任务的相关按钮
对话	用户可以用自然语言的方式，在 AI 对话部分完成相关操作
操作	用户可以通过勾选和拖动的方式，较为直观地操作待办任务
探索	用户可以在日历提供的线性时间空间中自由探索，查看和安排对应任务

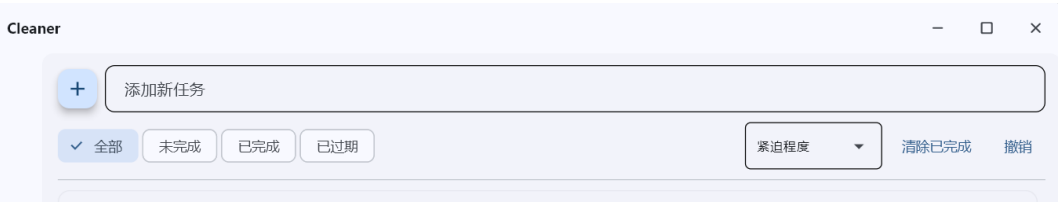
软件提供了主页面+4 个子功能页面的功能组织方式，用户可以通过切换/开关页面的方式进行探索。

3.3 认知层面

注意力

任务管理属于高频、短时、碎片化场景中的事务。因此软件中创建任务的输入框是最突出的。

输入框



分层次展示不同层级信息，随着用户的操作而逐级展开所需要的功能，而非一次性出现所有功能导致分散注意力。

点击输入框后的具体设置菜单



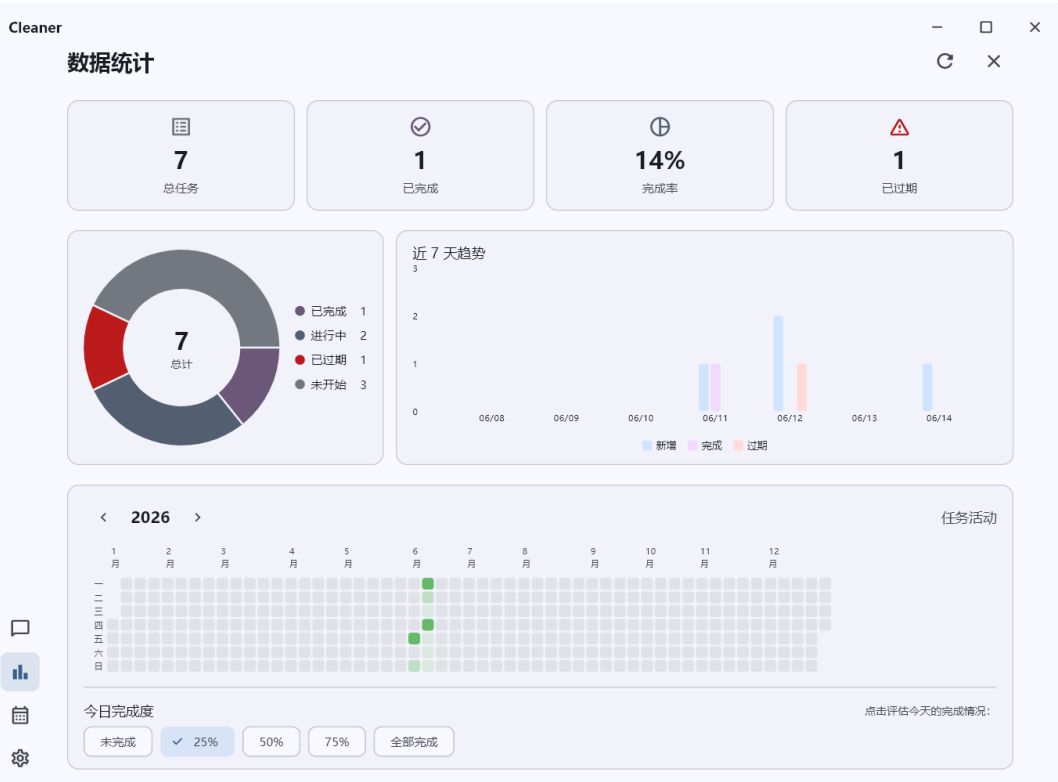
感知

任务通过文字、深浅、颜色、进度条等方式展示信息。用户大脑无需调用语言中枢，仅凭色彩冷暖和透明度即可在毫秒级"感知"任务的危急程度。

任务卡片



图表



记忆

- AI 自动记录最近对话的上下文记忆。
- 重新打开重写后的日期选择器时，系统会自动前置填入该任务原有的日期和重复模式。
- 用户不需要记住"我刚才对 AI 说了什么"或"这个任务原来定在几号"，系统代为记忆。

学习

提供 7 步基础教程。



阅读

- 提供多语言支持。
- AI 助手回复使用 Markdown 渲染，有效展示排版效果。

规划推理

- 提供全面详细的可选编辑项，辅助用户拆解自身需求，转化为标准化格式的待办任务。
- AI 助手也可辅助用户将直觉上的自然语言转换为待办任务。

创建任务菜单

The screenshot displays a task creation interface. At the top, there is a button labeled '+ 添加新任务' (Add New Task) and a date range '今天 - 06-18'. Below this is a calendar for June 2026, with the 14th and 18th highlighted. To the right of the calendar is a date range selector showing '2026-06-14 ~ 2026-06-18' and a note '点击选择日期，再点另一个变为持续' (Click to select a date, then click another to make it continuous). Below the date range selector are two time pickers for '06-14' and '06-18', each with a dropdown menu for hours and minutes. Below the time pickers is a '重复' (Repeat) section with a checked '不重复' (Do not repeat) button and other options: '每天' (Every day), '隔天' (Every other day), '每3天' (Every 3 days), '每7天' (Every 7 days), and '自定义' (Custom). At the bottom, there is a text input field '添加描述 (可选)' (Add description (optional)) and a set of filter buttons: '全部' (All), '未完成' (Not completed), '已完成' (Completed), and '已过期' (Expired). On the far right, there is a '紧迫程度' (Urgency) dropdown menu, a '清除已完成' (Clear completed) button, and a '撤销' (Undo) button.

3.4 认知框架

内部框架

- 用户产生创建待办任务的念头时，往往先想到事务的概念本身，再附加详细描述，正符合软件中以待办任务名为中心，附带其他描述属性的任务创建方式。
- 左侧滑出的聊天抽屉，符合物理世界中抽屉的开合逻辑。这符合用户将"AI 助手"视为一个"在旁边帮忙参谋的协同者"的心理预期，而不是一个阻断工作的模态屏障。

外部框架

- 待办软件本身主要功能就是辅助用户将规划具象化为应用软件对象，进行操作整理。
- 任务的自动化管理，如排序计算，长期任务完成情况的整理与统计，都在软件中进行实现。将一部分计算和记忆交给了软件本身进行处理，降低了思考和操作的负荷。

3.5 社会化交互

- 统计大屏引入了极具极客文化特征的 GitHub 风格 26 周热力图，以及高颜值的环状分布图和柱状趋势图。
- 高质量的数据可视化图表设计天然具备极强的可分享性，用户可以通过截图将连续的"深绿色打卡记录"发布到社交平台进行生活记录和分享。
- 本软件将原本枯燥的"划掉待办"，转化为了一幅代表个人意志力与长期主义的视觉画卷。用户在社交平台分享热力图的动作，实际上产生了社会化监督的作用，从而反向激励用户持续使用该软件。

3.6 情感化交互

- 界面采用极简风格设计，放弃了高饱和度的原色，转而采用低对比度的莫兰迪色系作为任务状态的区分，配合大半径弥散阴影和 12px 统一圆角，能够有效中和面对大量繁杂任务时产生的烦躁感，向用户传递出一种"克制、优雅、井然有序"的视觉情绪。
- 系统在设置页精心预设了多套 AI 助手性格（阿喵、阿汪、砖家、小冰）。它不仅是一个执行程序的冷冰机器，更是一位帮助用户温暖伙伴。

四、开发实现

4.1 关键技术决策

- **LLM 双重调用消除**：`llm_service.py:process()` 新增可选参数 `intent: PlannedTaskIntent | None`，调用方将首次 `plan()` 的结果传入，避免重复调用，响应延迟减少约 50%。
- **NLP 离线回退**：`nlp_task_parser.py` (411 行) 使用中文正则/关键词解析器，当 LLM 不可用（无 API Key 或调用失败）时自动降级，实现真正离线可用。
- **datetime 类型统一**：`TaskRecord.date` 和 `end_date` 从 `date` 改为 `datetime`，支持精确到小时/分钟的时间管理。`_parse_datetime()` 兼容 ISO 完整格式和纯日期格式。
- **连接缓存机制**：`storage/db.py` 的 `get_connection()` 按数据库路径缓存 SQLite 连接，通过 `SELECT 1` 检测有效性，避免频繁创建/销毁连接。
- **事务管理器**：`transaction()` 上下文管理器自动 `commit/rollback`，确保写操作的原子性。
- **UI 冻结修复**：引入 `_needs_resort` 标志，仅在任务列表真正变化时才触发 `before_update()` 排序。Card 级 `update()` 替代全局 `self.update()`，避免 IPC 往返引起的冻结。

4.2 核心模块实现

4.2.1 LLMService

- 使用 LangChain 的 `ChatOpenAI` + `ChatPromptTemplate` 进行工具选择
- `TaskPlan` Pydantic 模型定义 15 个字段的工具规划 (`tool`, `action`, `task\_name`, `date\_text`, `end\_date\_text`, `repeat\_days`, `repeat\_mode` 等)
- `PlannedTaskIntent` 数据类承载解析后的意图
- `tools()` 定义 8 个工具 (`create\_task` / `list\_tasks` / `update\_task` / `delete\_task` / `complete\_task` / `uncomplete\_task` / `plan\_tasks` / `help`)
- `\_memory` 列表 + `\_load_memory()` / `\_remember()` 实现持久化对话记忆
- `rebuild()` 方法支持运行时重配 LLM

4.2.2 TaskRecord 数据模型

```
@dataclass(slots=True)
class TaskRecord:
    id: int | None
    name: str
    date: datetime
    end_date: datetime | None
    description: str
    completed: bool
    repeat_days: int
    repeat_mode: str
    completed_dates: list[str]
```

4.2.3 ThemeManager

- 单例模式，管理 `theme_mode` (`light/dark/system`)、`seed_name`、`language`
- `apply(page)` 设置 `page.theme` / `page.dark_theme` / `page.theme_mode`
- 通过 `SettingRepo` 持久化偏好
- `AppColors` 类：语义颜色绑定 Material 3 token (`PANEL_BG`→`SURFACE_CONTAINER_LOW`, `BUBBLE_USER`→`PRIMARY_CONTAINER` 等)

4.2.4 i18n 多语言系统

- 190+ 翻译键，`MESSAGES` 字典结构 `{key: {"zh": ..., "en": ...}}`
- `t(key, *args)` 函数读取 `ThemeManager.language`，返回对应语言字符串，支持 `format(*args)`
- 语言切换回调触发 `\_rebuild_views()` 实时重绘所有 UI

4.2.5 自定义日期选择器

- 完全自定义日历控件，替代 Flet 原生 `DatePicker`
- 自动范围检测：无需显式"持续"开关，点第一个日期→start，点第二个→end
- 自适应时间行：单日期一行、同天持续两行、跨天持续两行
- 两列布局：左列日历网格 + 右列时间选择 + `extra\_controls`（如重复设置）
- 月份切换动画：`animate_position=200` 平滑过渡

4.2.6 系统通知服务

- 基于 `winotify` 发送 Windows 原生 toast
- `send(title, body, tag)` 自动跳过免打扰时段，按 tag 去重
- 免打扰逻辑支持跨午夜时段
- 设置通过 `SettingRepo` 持久化

4.2.7 数据存储层

模块	文件	职责
DB 连接	<code>db.py</code>	连接 <code>_connection_cache</code> 、 <code>transaction()</code> 上下文管理器、建表+自动迁移
任务仓储	<code>task_repo.py</code>	核心数据结构 8 字段 CRUD
设置仓储	<code>setting_repo.py</code>	管理 settings 表
评估仓储	<code>daily_assessment_repo.py</code>	管理 <code>daily_assessments</code> 表

4.3 程序运行流程

4. `before_main()`：从 SQLite 恢复窗口尺寸/位置，Win32 API 获取多显示器信息，智能检测窗口是否在可见屏幕上
5. `main()`：设置窗口属性 → 初始化 `SettingRepo` / `ThemeManager` / `LLMConfigManager` / `NotificationService` → 注册窗口事件回调 → 根据 onboarding 条件决定显示教程或主界面
6. `_show_main_app()`：创建 `TodoApp`（含 `LLMService`、`TaskService`、`StatsView`、`CalendarView`、`SettingsView`）

4.4 优化历程

该项目经过 **50 次迭代优化**，覆盖以下维度：

优化维度	次数	优化关键内容
AI 系统	5	双重调用消除、NLP 回退、LLM 记忆持久化、配置管理器、预设性格
UI/UX	18	聊天抽屉、视觉美化、气泡动画、Toast、快捷键、空状态、卡片动画、hover 效果、阴影圆角优化
数据存储	3	连接缓存、事务管理器、自动迁移
任务模型	5	datetime 类型统一、重复任务、进度条、到期时间、拖拽圆角
可视化	4	统计页面(flet-charts)、热力图、日历视图、主题适配
多语言	1	翻译键、实时刷新
窗口管理	3	位置记忆、多显示器支持、全屏/最大化状态、before_main 预加载
通知系统	2	winotify 原生通知、异步调度器、免打扰/去重
教程	2	7 步交互式引导、统一卡片高度、设置页入口
性能	3	UI 冻结修复
兼容性	2	API 修复、日期选择器两列重构

五、实验评估

5.1 评估目标

- **易学性验证**：新用户是否能够快速理解软件功能并正确使用？
- **AI 助手功能验证**：自然语言对话是否比传统表单输入更受欢迎？
- **情感化设计**：界面设计和数据统计是否为用户提供了正向情绪价值？
- **用户满意程度**：用户是否愿意长期使用该软件？

5.2 被试者招募

- **样本规模**：N=6 人

- **用户构成：**
- 职场人士：2 名
- 大学生：4 名
- **招募要求：**熟悉 Windows 桌面操作系统基本操作，无严重视力障碍。

5.3 实验流程

(1) 任务执行

提供配置好的软件，让用户完成以下任务：

编号	任务描述
Task1	请尝试添加一个任务：明晚 8 点，提醒我打电话。你可以用任何你觉得方便的方式。
Task2	你决定每天背 50 个单词，请在软件里把这件事安排好，并标记今天的份已经完成。
Task3	将待办任务按照日期排列，将打电话的任务拖拽到最上方。
Task4	请查看自己过去一个月的任务完成情况。
Task5	请将软件改为深色模式，并把 AI 助手的性格换成"阿喵"。

(2) 问卷调查

在完成任务后，让用户自由探索软件功能，之后填写问卷。

每个问题选项分为"非常不认同""有点不认同""中立""有点认同""非常认同"

问卷题目：

7. 我认为软件功能易于上手使用
8. 我可以理解不同页面的功能含义
9. 我愿意频繁使用 AI 助手辅助处理任务
10. 我认为 AI 助手辅助处理任务比手动操作更加高效
11. 我认为界面应该更加简单，呈现更少的元素
12. 我可以将软件外观调整为我需要的样式
13. 如果长期使用，我会将数据统计页面截图分享到社交平台
14. 我愿意长期使用这个软件

5.4 实验结果

任务完成情况：所有被试者均在 5 分钟内顺利完成了 5 项任务，任务成功率达 **100%**。

问卷结果：

题目	非常不认同	有点不认同	中立	有点认同	非常认同
软件功能易于上手使用	0	0	0	1	5
理解不同页面的功能含义	0	0	0	0	6
愿意频繁使用 AI 助手	2	2	2	0	0
AI 助手比手动更高效	4	2	0	0	0
界面应更简单	0	4	2	0	0
可调整为所需样式	0	0	0	3	3
会截图分享到社交平台	0	2	3	1	0
愿意长期使用	0	1	4	1	0

5.5 分析结果

对于最初提出的 4 点研究目标：

1. 易学性 — 非常成功

基础界面设计与可用性非常成功。所有任务能被新用户较快上手完成，界面易学性获得良好评价。系统的信息架构非常清晰，外部特征完全吻合用户的内部心理模型，实现了真正的“零门槛上手”。

2. AI 助手功能 — 未达预期

用户一致拒绝将 AI 作为日常操作的主要手段，认为其效率远低于手动界面操作。比如对于“明晚 8 点提醒我打电话”这种简单任务，在设计优良的 GUI 中只需输入任务名称后点击几下。而要求用户通过键盘打字输入一段完整的自然语言，不仅存在物理上的“打字成本”，还需要用户组织语言、等待 AI 响应，整体耗时反而更长。

但这也反向证明了 Cleaner 的基础手动操作界面足够高效流畅，易于被用户使用。

3. 情感化设计 — 部分达成

用户对界面的视觉密度满意，无需进一步简化。但用户对于截图分享持保留态度，待办数据（即使是经过图形化的热力图）本质上属于较为隐私的个人生活轨迹。用户内心缺乏将其公开展示的“自我呈现”动力，导致社会化交互目标未达成。

4. 长期使用意愿 — 中立

用户整体长期使用意愿呈现中立。在当前的软件生态中，如果 Cleaner 的"AI"标签不能真正提升效率，它就仅仅是一个"长得比较好看的普通待办工具"。在面对市场上的其他成熟竞品时，缺乏足够的转换动力，导致长期使用意愿平淡。

六、项目总结

本项目从零开始设计并实现了一款融合 AI 语义理解的桌面待办管理应用。经过 50 余次迭代开发，最终完成了一个功能完整、体验流畅的产品原型。

1. 技术成果

在技术层面，项目实现了多项有价值的技术实践：基于 LangChain + DeepSeek 的中文自然语言任务解析系统，支持 8 种工具调用和 15 个字段的结构化意图输出；设计了 NLP 离线回退机制，确保在网络异常时应用仍可正常运行；实现了完整的数据持久化方案，包括 SQLite 存储、连接缓存和事务管理；构建了自定义日历控件、GitHub 风格热力图等高复杂度 UI 组件；完成了系统通知、多语言国际化、窗口状态记忆等桌面应用必备能力。整体采用 UI-Service-Storage 三层架构，保证了代码的可维护性。

2. 设计实践

在人机交互设计方面，项目以"降低输入成本、提升信息感知效率"为核心目标，探索了 AI 对话与传统 GUI 操作的融合方式。实验结果表明，精心设计的手动操作界面在效率上优于自然语言输入，这为"AI 优先"的设计范式提供了有价值的反面参考。同时，莫兰迪色系、弥散阴影、进度条等视觉设计元素获得了用户的积极反馈，验证了情感化设计在工具类软件中的可行性。

3. 经验与反思

通过本次项目，团队获得了以下关键认知：其一，AI 能力的引入应当瞄准传统交互难以解决的痛点（如复杂语义理解、批量任务规划），而非替代已经高效的简单操作；其二，用户对隐私数据的分享意愿低于预期，数据可视化的社交传播设计需要更审慎的考量；其三，桌面应

用的体验打磨涉及大量细节（窗口管理、动画流畅度、快捷键等），这些“看不见的工作”往往是决定产品质感的关键因素。

总体而言，Cleaner 作为一个课程实践项目，在技术实现的完整度和交互设计的思考深度上达到了预期目标，同时也为团队积累了宝贵的全栈开发与用户研究经验。